

VITYARTHI PROJECT



RANDOM MOBILE NUMBER ASSIGNER

NAME-HARSHIT

REGISTRATION NUMBER-26BCE10635

PROBLEM STATEMENT

The problem is to solve the problem with help of programming language :

- **User Data Management:**
Store user details such as name, age, etc.
- **Random Mobile Number Assigner:**
Automatically generate a random 10 digit mobile number
- **User search:**
Prevent duplicate mobile number
- **User Management:**
Provide search, display, update, delete, and count operations.
- **Data Persistence:**
Save user data permanently in a text file and loading existing data when the program starts.
- **Python Implementation:**
Implement the system using python concepts.

INTRODUCTION

Random Mobile Number Assigner is final project for Vityarhi Python Essentials course. This project is a python-based command line project which is developed using the concepts of Python Essentials teachings. The main purpose of this project is to assign random mobile numbers to users and manage their information through a simple menu-driven interface. This project started from 23 August 2026 to 23 September 2026. This is done during the first semester before advancing into the industrial program. Therefore, it is vital to complete the project to receive a final grade in the effort to put in.

The project uses Python dictionaries to store user information and file handling to save the data in a text file named user.txt. It loads data from user.txt file and can avoid error in case of restarting the program also.

The Vityarhi Project also gives students individual ability and confidence to complete a task under supervision of lectures. With this, Student can learn problem-solving skills in areas of designing analysis, fabrication and testing as well learn to do a complete formatted which is important to future technical writing

DESIGN, DECISION & RATIONALE

1. Dictionary: Fast and simple user data management.
2. Random module: Generates mobile number automatically.
3. Duplicate Check: Prevents repeated mobile numbers.
4. Menu Interface: Provide simple command line instructions.
5. Functions: Separates different operations for better operations.
6. File: Save and load data permanently.
7. Validation: Handles invalid inputs and duplicate users.

SYSTEM ARCHITECTURE AND DESIGN

1. Programming Language: Python
2. Interface: Command Prompt
3. Storage: users.txt
4. Module: random
5. Load File: Users.txt
6. Try-except: Handle errors

FUNCTIONAL REQUIREMENTS

1. Generating the mobile number:

```
# 1. Generate Random Mobile Number

def genmob():
    while True:
        firstdig=random.choice("6789")
        remainingdig=""
        for i in range(9):
            dig=str(random.randint(0,9))
            remainingdig=remainingdig+dig

        mob=firstdig+remainingdig

        if mob not in users.values():
            return mob
```

2. Assigning the mobile number:

```
=====2.Assign Random Mobile Number=====

def assignnum():
    print("\nAssigning a mobile number to you")

    name=input("Enter your name= ").strip()
    if(name==""):
        print("Name cannot be empty")
        return
    if(name in users):
        print("Name is already registered")

    try:
        age=int(input("Enter the age= "))

        if(age<=0 and age>120):
            print("\nEnter valid age")
            return
    except ValueError:
        print("Age must be a number")
        return

    mob=genmob()

    users[name]={"age":age,"mob":mob}

    print("\nRegistration sucessfully")
    print("Name=",name)
    print("Age=",age)
    print("Assigned Mobile Number=",mob)
```

3. Searching user:

```
#-----  
#=====3.Search User=====  
#-----  
  
def searchuser():  
    print("\n----- Searching User-----")  
  
    name=input("Enter the name= ")  
  
    if(name in users):  
        print("Name is=",name)  
        print("Age is=",users[name]["age"])  
        print("Mobile number is",users[name]["mob"])  
  
    else:  
        print("User not found")
```

4. Displaying User:

```
#----- Display all users-----  
#-----  
  
def displayusers():  
    print("All registered users are following: ")  
  
    if(len(users)==0):  
        print("No user registered in database")  
        return  
  
    count=1  
  
    for name,details in users.items():  
        print("\nUsers",count)  
        print("Name",name)  
        print("Age",details["age"])  
        print("Mobile number",details["mob"])  
  
        count+=1
```

5.Updating User:

```
-----Update user age-----
```

```
def updateage():
    print("\n----Updateding your Age")

    name=input("Enter the name=").strip()
    if(name not in users):
        print("User not found")
        return
    try:
        age=int(input("Enter the age="))
        if(age<0 and age>120):
            print("Enter a valid age")
            return
    except ValueError:
        print("Age must be valid")
        return
    users["age"]=age

    print("Age updated sucessfully")
```

7. Deleting User:

```
#-----DeleteUser-----
#
def deluser():
    print("\n-----Updating user-----")

    name=input("Enter the name=")

    if name in users:
        del users[name]
        print("User deleted sucessfully")

    else:
        print("User not found")
```

7.Count users:

```
#-----Count users-----  
#-----  
  
def countusers():  
    print("\n Total numberof users are= ",len(users))  
  
#-----  
#7.=====Save users=====
```

8. Saving user's information:

```
#-----
#7.=====Save users=====
#-----
def saveusers():
    with open("Users.txt","w") as file:
        for name,details in users.items():
            age=details["age"]
            mob=details["mob"]

            file.write(name+"|"+str(age)+"|"+mob+"\n")

#-----
#=====
#-----
```

9.Load Users:

```
#-----  
#=====
```

```
def loadusers():  
    try:  
        with open("Users.txt","r") as file:  
            for line in file:  
                line=line.strip()  
  
                if(line==""):  
                    continue  
                data=line.strip("|")  
  
                if(len(data)==3):  
                    name=data[0]  
                    age=int(data[1])  
                    mob=data[2]  
  
                    users[name]={"age":age ,"mob": mob}  
    except FileNotFoundError:  
        print("No previous data found")  
    except ValueError:  
        print("Invalid data in file")
```

DESIGN DAIGRAM

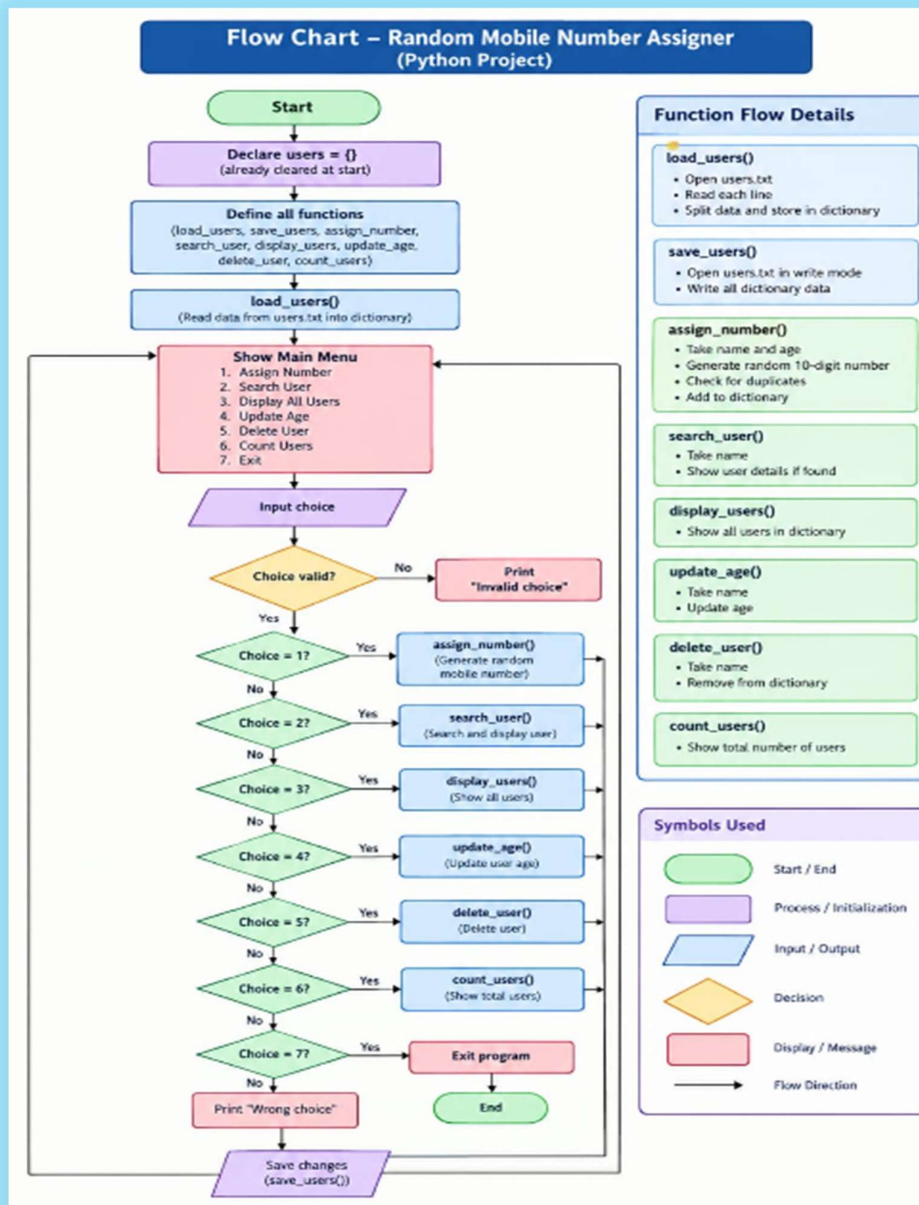


FIGURE 1

IMPLEMENTATION

1. **User Input:** Takes name and age from the user.
2. **Data Storage:** Stores user details in a dictionary.
3. **Number Generation:** Using random module.
4. **Duplicate Checking:** Checks whether generated number exists or not
5. **User operations:** Implements search, display, update, delete and count functions.
6. **Menu Control:** Uses if-else and a While loop to manage user's requests.
7. **Data Persistence:** Loads saved data at startup and saves modification to the file.

OUTPUT

```
Signed) ; (& c:\Users\harsh\OneDrive\Desktop\Python\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\harsh\OneDrive\Desktop\Python> & c:\Users\harsh\OneDrive\Desktop\Python\.venv\Scr
ipts\python.exe "c:/Users/harsh/OneDrive/Desktop/Python/py/Studentcalculator/VITYARTHI PROJECT/RANDOM
MOBILE NUMBER ASSIGNER"
=====
RANDOM MOBILE NUMBER ASSIGNER=====
Choose your request from given list
Assigning random mobile number code: 1
Search user code: 2
Display all user code: 3
Update age code: 4
Delete user code: 5
Count user code: 6
Exit code: 7
Enter your request code=1

Assigning a mobile number to you
```

```
Enter your name= Harshit
Enter the age= 24

Registration sucessfully
Name= Harshit
Age= 24
Assigned Mobile Number= 8121482855
Want to proceed further? (yes/no): no
REQUEST COMPLETED
(.venv) PS C:\Users\harsh\OneDrive\Desktop\Python> █
```

TESTING APPROACH

1. **Input validation:** Tested valid and invalid user inputs.
2. **Number Testing:** Verified generation of 10 digit number.
3. **Duplicate Testing:** Checked prevention of duplication.
4. **Search Testing:** Tested age updates for existing user.
5. **Delete Testing:** Verified user deletion
6. **Fiel Testing:** Verified user deletion
7. **Menu Testing:** Tested all menu options and exit functionality.

CHALLENGES FACED

1. **Random Mobile Number Generation:** Generating number without repetition.
2. **File Handling:** Correctly reading and writing data into Users.txt .
3. **Data synchronization:** Keeping dictionary data and file data consistent.
4. **Input Handling :** Managing invalid or unexpected data consistent.
5. **Program Flow:** Controlling repeated menu operations and exit conditions.
6. **Debugging:** Identifying and fixing errors during development.

LEARNING AND TAKEAWAYS

1. **Python Fundamentals:** Strengthened understanding of core python concepts.
2. **Dictionary:** Learned how to store and manage structured data.
3. **Functions:** Learned to divide program into sub-programs.
4. **File Handling:** Learned to store and retrieve data using text files.
5. **Random Module:** Learned to generate random values programmatically.
6. **Program Design:** Learned to develop a complete menu-driven application.

FUTURE

ENHANCEMENTS

1. **Graphical Interface:** Develop a user- friendly interface.
2. **Database management:** Replace the text file with a database.
3. **Input Validation:** Add stronger validation for user input.
4. **User authentication:** Add secure login functionality.
5. **Data Export:** Allow options to export data to CSV or excel sheet.

CONCLUSION

At last, We can conclude that our project Random Mobile Number Assigner is successfully completed under the guidance of our professor Dr. Prabu Kanna G who helped us in completing this project.

We can also infer that this project helped us to improve our problem-solving skills and understanding the project from end to end.

THANK YOU

HARSHIT

26BCE10635

